

Programação Funcional

Padrões Simples e Definição de Operadores em Haskell

Vladimir Oliveira Di Iorio
DPI - UFV

Operadores

- A linguagem Haskell contém vários operadores, como: $+$, $-$, $*$, $/$, ``div``, ``mod``, $\wedge$, etc.
- Operadores são *infixos*, o que significa que são escritos entre os seus argumentos.
- Parêntesis podem ser usados em qualquer operação: $((4+8)*3)+2$
- Propriedades que auxiliam na eliminação de parêntesis: **associatividade** e **prioridade** de operadores.

Associatividade

- Uma operação op é associativa se
$$(x \text{ op } y) \text{ op } z = x \text{ op } (y \text{ op } z)$$
- Operações associativas dispensam parêntesis.
- Uma operação não associativa é classificada como:
 - associativa à esquerda; ou
 - associativa à direita.

Associatividade

Adição (+) é associativa.

Main> 4 + 2 + 1

7

Main> (4 - 2) - 1

1

Main> 4 - (2 - 1)

3

Main> 2 - 1

1

Ma

51

Subtração (-) é associativa.

Subtração (-) é

Exponenciação é associativa à direita.

Precedência de operadores

- A associatividade define a ordem de computação de expressões envolvendo mais de uma ocorrência de um mesmo operador. Ex: 2^3^2
- E quando operadores diferentes são utilizados?
- Por exemplo:
 $2+3*4$
 3^4*2
- Alguns operadores têm **precedência** maior do que outros.

Precedência de operadores

Main> 2 + 3 * 4

14

Main> 2 ^ 3 * 4

32

Operador * tem precedência 7.

Operador ^ tem precedência 8.

Precedência de operadores

```
Main> 2 + 3 * 4
14
Main> 2 ^ 3 * 4
32
Main> fat 4 + 1
????????????????
```

```
1 fat :: Int -> Int
2 fat n
3   | n <= 0      = 1
4   | otherwise   = n * fat (n-1)
```

Precedência de operadores

```
Main> 2 + 3 * 4
```

```
14
```

```
Main> 2 ^ 3 * 4
```

```
32
```

```
Main> fat 4 + 1
```

```
25
```

Aplicação de função tem precedência máxima.

Tabela de associatividade e precedências pode ser encontrada no Apêndice E do livro-texto.

```
1 fat :: Int -> Int
2 fat n
3   | n <= 0      = 1
4   | otherwise   = n * fat (n-1)
```

Precedência de operadores

```
Main> fat (-1)  
1
```

```
1 fat :: Int -> Int  
2 fat n  
3   | n <= 0      = 1  
4   | otherwise   = n * fat (n-1)
```

Precedência de operadores

```
Main> fat (-1)
```

```
0
```

```
Main> fat -1
```

```
ERROR - Illegal class constraint in  
inferred type
```

```
1 fat :: Int -> Int  
2 fat n  
3   | n <= 0      = 1  
4   | otherwise   = n * fat (n-1)
```

Definindo Operadores

- Haskell permite a definição de novos operadores.
- Os nomes dos operadores são formados por:

! # \$ % & * + . / < = > ? @ \ ^ _ | : - ~

- Restrições:

- nomes não podem começar por "."
- "-"
- símbolos

Define

Define o
associativ

Define operador como
associativo à direita.

- Mudar as prioridades e a precedência:

infix

infixl

infixr

Definindo Operadores

```
1 infixl 7 &&&
```

Define operador &&& como associativo à esquerda, com predência 7.

Definindo Operadores

```
1 infixl 7 &&&
2
3 (&&&) :: Int -> Int -> Int
4 a &&& b
5   | a > b      = a
6   | otherwise  = b
```

```
Main> 10 &&& 20
20
```

Definições com Padrões

- As definições de funções vistas até agora consistem de uma única equação.
- É possível definir uma função usando várias equações.
- As equações devem especificar como a função se comporta quando aplicada a determinados **padrões**.
- Os padrões mais simples são variáveis e constantes.

Definições com Padrões

- Versão anterior - uma equação condicional:

```
1 totalSales :: Int -> Int
2 totalSales n
3   | n == 0      = sales 0
4   | otherwise   = totalSales (n-1) + (sales n)
```

- Nova versão - usando padrões:

```
1 totalSales 0 = sales 0
2 totalSales n = totalSales (n-1) + (sales n)
```

Definições com Padrões

- 1 $\text{totalSales } 0 = \text{sales } 0$
- 2 $\text{totalSales } n = \text{totalSales } (n-1) + (\text{sales } n)$

- A primeira equação é aplicada somente ao valor 0.
- A segunda se aplica a todos os outros casos.
- Regra geral: utilizar a primeira equação que "casa" com os argumentos, na ordem especificada.
- Um argumento a casa com um padrão p se:
 - p é uma constante e a é igual a p ; ou
 - p é uma variável.

Definições com Padrões

- O padrão "_" casa com qualquer argumento.
- É usado quando o valor do argumento não é necessário no lado direito da equação.

```
1 is Zero :: Int -> Bool
2 isZero 0    = True
3 isZero _    = False
```

Definições com Padrões

- Exemplo envolvendo padrões e equação condicional:

```
1 fib :: Int -> Int
2
3 fib 0 = 0
4 fib 1 = 1
5 fib n
6   | n > 1      = fib (n-1) + fib (n-2)
7   | otherwise  = 0
```

Programando com Booleanos

- Os valores booleanos são representados pelas constantes `False` e `True`, do tipo `Bool`.
- Operadores:
`&&` ("e" lógico) `||` ("ou" lógico) `not` (negação)
- Semântica dos operadores:

t1	t2	t1 && t2	t1 t2
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

t1	not t1
T	F
F	T

Programando com Booleanos

- Uma definição para "not":

```
1 meuNot :: Bool -> Bool
2 meuNot True = False
3 meuNot False = True
```

Programando com Booleanos

- "ou exclusivo" - retorna True se exatamente um dos argumentos for True:

```
1 exOr :: Bool -> Bool -> Bool
2 exOr x y = (x || y) && not (x && y)
```

- Definição usando padrões:

```
1 exOr False x = x
2 exOr True x = not x
```

Programando com Booleanos

- Construir uma função `isZeroDay` que, dado o número de um dia, retorne:
 - `True`, se o número de unidades vendidas (função `sales`) no dia é zero;
 - `False`, caso contrário.

```
1 isZeroDay :: Int -> Bool
2 isZeroDay n
3   | sales n == 0    = True
4   | otherwise       = False
```

Programando com Booleanos

```
1 isZeroDay :: Int -> Bool
2 isZeroDay n
3   | sales n == 0      = True
4   | otherwise         = False
```

- Quando a expressão `sales n == 0` tem valor `True`, a função retorna `True`.
- Quando a expressão `sales n == 0` tem valor `False`, a função retorna `False`.

Programando com Booleanos

```
1 isZeroDay :: Int -> Bool
2 isZeroDay n
3   | sales n == 0      = True
4   | otherwise         = False
```

- Simplificação:

```
1 isZeroDay :: Int -> Bool
2 isZeroDay n = (sales n == 0)
```

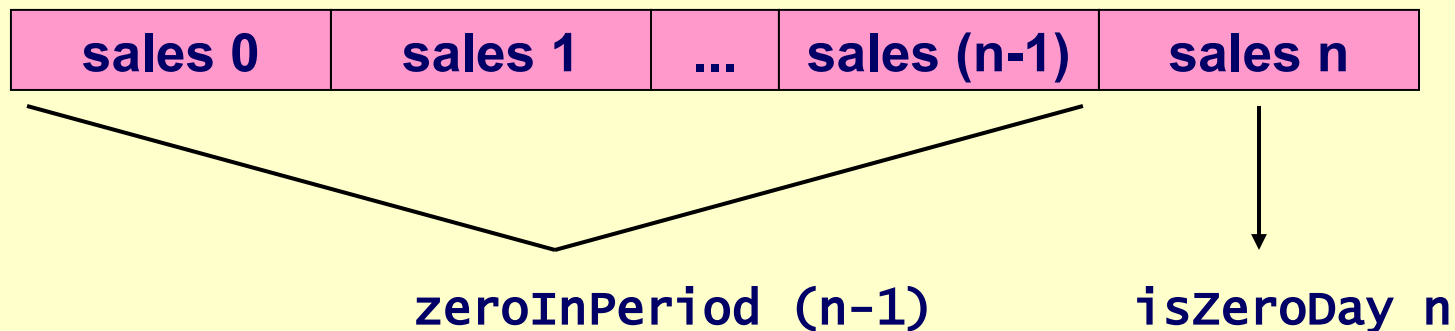
Programando com Booleanos

- Construir uma função `zeroInPeriod` que, dado o número de um dia, retorne:
 - `True`, se não foi vendida nenhuma unidade do produto em algum dia do período $0, \dots, n$;
 - `False`, caso contrário.

```
1 zeroInPeriod :: Int -> Bool
2
3 zeroInPeriod 0 = isZeroDay 0
4 zeroInPeriod n = ??????????????????
```

Programando com Booleanos

```
1 zeroInPeriod :: Int -> Bool
2
3 zeroInPeriod 0 = isZeroDay 0
4 zeroInPeriod n =
```



Programando com Booleanos

```
1 zeroInPeriod :: Int -> Bool
2
3 zeroInPeriod 0 = isZeroDay 0
4 zeroInPeriod n = zeroInPeriod (n-1) || isZeroDay n
```



`zeroInPeriod = zeroInPeriod (n-1) || isZeroDay n`

Caracteres e Strings

- Char : tipo de Haskell associado a caracteres.
- Caracteres individuais são inseridos em aspas simples. Exemplos: 'd', '3'.
- Caracteres especiais:

<code>'\t'</code>	tab
<code>'\n'</code>	new line
<code>'\\'</code>	barra invertida
<code>'\''</code>	aspa simples
<code>'\"'</code>	aspa dupla
<code>'\97'</code>	caractere 'a'

Caracteres e Strings

- Haskell utiliza a codificação padrão ASCII para caracteres.
- As funções `chr` e `ord` são usadas para conversões.

```
Main> chr 97
'a'
Main> ord 'A'
65
```

Caracteres e Strings

- Conversão de letra minúscula para maiúscula:

```
1 offset = ord 'A' - ord 'a'
2
3 capitalize :: Char -> Char
4 capitalize ch = chr (ord ch + offset)
```

- Verificar se um caractere é um dígito:

```
1 isDigit :: Char -> Bool
2 isDigit ch = ('0' <= ch) && (ch <= '9')
```

Caracteres e Strings

- Strings de caracteres pertencem ao tipo `String`.
- São inseridas entre aspas duplas. Exemplos:
 - `"abcdef"`
 - `"uma linha\noutra linha"`
 - `""`
 - `"O caractere \'a\' : \97"`
- Strings são concatenadas usando o operador `++`.

Caracteres e Strings

- Haskell permite que novos tipos sejam criados, usando a palavra-chave `type`.
- O tipo `String` é, na realidade, uma lista de caracteres:

```
type String = [Char]
```

- As operações que manipulam listas podem ser usadas também para strings.

Números de Ponto Flutuante

- Números de ponto flutuante são representados, em Haskell, pelos tipos `Float` e `Double`. Exemplos:

3.141592

-1.2345

987.654

- Haskell usa também notação científica:

$$9.87654e02 = 9.87654 * 10^2 = 987.654$$

$$31415.92e-4 = 31415.92 * 10^{-4} = 3.141592$$

Números de Ponto Flutuante

- Haskell oferece uma série de funções que atuam sobre números de ponto flutuante.

```
Main> sin (pi/2)
1.0
Main> 1.1 ^ 10
2.59374
Main> 1.1 ^ 10
2.59374
Main> 1.1 ** 10.0
2.59374
Main> sqrt 2
1.41421
```

Números de Ponto Flutuante

```
Main> round 2.49
2
Main> ceiling 2.49
3
Main> floor 2.49
2
Main> log 2
0.693147
main> logBase 2 16
4.0
```